

Refactoring Report-HW7

Lawrence Qian

October 2025

1 Introduction

The original code was implemented as a Jupyter notebook for an academic assignment on batch normalization and dropout in neural networks. While functional, the notebook followed an exploratory workflow: cells mixed configuration, computation, visualization, and debugging code.

The goal of this refactoring was to transform that notebook into a clean, modular, and reproducible project that follows modern software engineering principles, including:

Modularity — encapsulating reusable logic into Python modules (`data.py`, `model.py`, `bn_experiments.py`, etc.).

Reproducibility — consistent seeding and configuration management (`config.py`).

Readability and Maintainability — unified logging, clear naming conventions, and elimination of redundant code.

Scalability — structured training pipelines and evaluation logic for both NumPy and PyTorch implementations.

The original notebook served its teaching purpose but blended exploratory analysis with implementation details, which makes it harder to maintain, test, and extend. This refactor re-structures the project into focused modules (configuration, data, models, training, and utilities) and documents the engineering choices. The goals are:

- **Preserve pedagogical clarity:** keep simple, self-contained examples and short “part” entry points that map to the original notebook cells.
- **Improve reproducibility:** centralize experiment configuration and deterministic seeding so results can be reproduced exactly.
- **Increase maintainability:** enforce consistent style (PEP 8), remove duplicated code, and split responsibilities across small modules.
- **Enable engineering workflows:** add structured logging, checkpointing, and a modular training API so the code can be tested, profiled, and integrated into CI or larger experiments.

The refactor follows software-engineering and ML-engineering best practices while retaining all examples and experiment code needed for teaching.

2 Code Comparison

2.1 Part1—Setup

```
import random, numpy as np, torch
np.random.seed(0)
torch.manual_seed(0)
print("Start")
```

Listing 1: Original

```
cfg = part1_setup() # calls set_seed(cfg.seed) and configure_logging()
```

Listing 2: Refactored

Commentary

- Centralized configuration, consistent seeding, structured logging.

2.2 Part2—Data loaders

```
def findFiles(path): ...
all_categories = ...
```

Listing 3: Original

```
loaders = part2_dataloaders(cfg)
```

Listing 4: Refactored

Commentary

- All file parsing and dataset split is encapsulated
- Returning dataloaders & metadata.

2.3 Part3—Inspect a batch

```
X, y = next(iter(train_loader))
X.shape
```

Listing 5: Original

```
indices , lengths , labels = part3_inspect_batch(loaders)
```

Listing 6: Refactored

Commentary

- Standardized batch format (padded indices, lengths, label) and consistent logging.

2.4 Part4—Model build

```
model = RecurrentClassifier(...)
model.cuda()
```

Listing 7: Original

```
model = part4_build_model(cfg , n_classes=meta["n_classes"])
```

Listing 8: Refactored

Commentary

- Single function builds model with config; device handling inside.

2.5 Part5—Initialize explicit params

```
# they often directly manipulated W tensors
W1 = model.rnn.weight_hh_l0
```

Listing 9: Original

```
# Inspect params via model.parameters() if needed; but avoid manual modification
```

Listing 10: Refactored

Commentary

- Avoid direct parameter hacks; if needed, provide a documented API.

2.6 Part6—Training loop

```

for epoch in range(n):
    for X,y in train_loader:
        optimizer.zero_grad()
        out = model(X)
        loss.backward()
        optimizer.step()

```

Listing 11: Original

```

res = part5_train(cfg, loaders)

```

Listing 12: Refactored

Commentary

- Extracts training loop for reuse, testing, logging, checkpointing.

2.7 Part7—Evaluation

```

val_acc = evaluate(model, val_loader)
print(val_acc)

```

Listing 13: Original

```

val_loss, val_acc = part6_evaluate(res, loaders)

```

Listing 14: Refactored

Commentary

- Uses the centralized eval function and consistent metric computation.

2.8 Part8—Hyperparameter sweep

```

for hd in [64,128]:
    for rnn in ['gru', 'lstm']:
        ...

```

Listing 15: Original

```

res_sweep = part9_sweep(cfg, loaders, rnn_types=['gru', 'lstm'], hidden_dims=[64,

```

Listing 16: Refactored

Commentary

- Encapsulates sweep & returns organized results dictionary.

2.9 Part9—Plotting

```
plt.plot(train_losses); plt.plot(val_losses)
```

Listing 17: Original

```
part10_plot_history(res, out_prefix="run1")
```

Listing 18: Refactored

Commentary

- Plotting encapsulated; safe for headless environments (plots saved to disk).

2.10 Part10—Save/load checkpoint

```
torch.save(model.state_dict(), 'model.pth')  
model.load_state_dict(torch.load('model.pth'))
```

Listing 19: Original

```
part7_save(res, "checkpoints/final.pt")  
ckpt = part7_load("checkpoints/final.pt")
```

Listing 20: Refactored

Commentary

- Adds directory safety, includes history and cfg in checkpoint, consistent load/save API.

2.11 Part11—Smoke test & inference

```
# single-step test
```

Listing 21: Original

```
part11_smoke_test(cfg)  
# classification:  
prediction = part8_classify_name(res["model"], "Satoshi", cfg, loaders["dataset_1"])
```

Listing 22: Refactored

Commentary

- Provides a reproducible smoke-test and a ready infer function.

3 Code Decomposition

`config.py` centralizes all tunable parameters and experiment-level settings. By consolidating dataset paths, model dimensions, optimization hyperparameters, and runtime options in a single dataclass, experiments become easier to launch, compare, and reproduce. Use this file to declare defaults and override them for specific runs.

```
# config.py
"""
Experiment configuration for the RNN name classifier project.
"""

from dataclasses import dataclass, asdict
from typing import Optional, List, Dict, Any

@dataclass
class Config:
    # Data
    data_dir: str = "data/names"           # local folder with per-category files (o
    batch_size: int = 128
    num_workers: int = 0

    # Model
    embedding_dim: int = 32
    hidden_dim: int = 128
    rnn_type: str = "gru"                   # "gru" or "lstm" or "rnn"
    num_layers: int = 1
    dropout: float = 0.0

    # Training
    lr: float = 1e-3
    weight_decay: float = 0.0
    num_epochs: int = 10
    seed: int = 0
    device: str = "cuda"                    # "cpu" or "cuda"
    log_interval: int = 100

    # Others
    output_dir: str = "checkpoints"
    max_name_length: int = 100

    def to_dict(self) -> Dict[str, Any]:
        return asdict(self)
```

Listing 23: `config.py`

`data.py` encapsulates data creation and loading. It provides a fast synthetic dataset generator for quick experiments and a clear placeholder for real data loaders (e.g., CIFAR) so the rest of the codebase can rely on a single dataloader API.

```
# data.py
"""
Data-loading utilities for the names dataset.
Assumes the original dataset layout where each category (e.g. 'Italian.txt') contains
Provides a PyTorch Dataset wrapper NameDataset and helper to build DataLoaders.
"""

from typing import Dict, List, Tuple, Optional
import os
import glob
import torch
from torch.utils.data import Dataset, DataLoader
from utils import unicode_to_ascii, name_to_indices, indices_to_tensor, ALL_LETTERS
import numpy as np

def find_files(path_pattern: str) -> List[str]:
    return glob.glob(path_pattern)

def load_names_by_category(data_dir: str) -> Dict[str, List[str]]:
    """
    Load names from files in data_dir. Each file becomes one category.
    Returns dict category -> list of names (ascii-cleaned).
    """
    files = find_files(os.path.join(data_dir, "*.txt"))
    categories = {}
    for fn in files:
        category = os.path.splitext(os.path.basename(fn))[0]
        with open(fn, encoding='utf-8') as f:
            lines = [unicode_to_ascii(line.strip()) for line in f if line.strip()]
            if lines:
                categories[category] = lines
    return categories

class NameDataset(Dataset):
    """
    Dataset of (name_indices_tensor, category_index). Each name is variable length.
    We'll return a dict {"indices": Tensor(Long, L), "length": int, "label": int}
    The DataLoader should use a collate_fn to pad sequences.
    """
    def __init__(self, categories: Dict[str, List[str]], max_length: int = 100):
```

```

        # Build category -> idx mapping
        categories_list = sorted(categories.keys())
        self.category_to_idx = {cat: i for i, cat in enumerate(categories_list)}
        self.idx_to_category = {i: cat for cat, i in self.category_to_idx.items()}
        self.max_length = max_length
        # Flatten data
        self._data = []
        for cat, names in categories.items():
            label = self.category_to_idx[cat]
            for name in names:
                inds = name_to_indices(name, max_length=self.max_length)
                if len(inds) == 0:
                    continue
                self._data.append((inds, label))
    def __len__(self):
        return len(self._data)
    def __getitem__(self, idx):
        inds, label = self._data[idx]
        return {"indices": torch.tensor(inds, dtype=torch.long),
                "length": len(inds),
                "label": torch.tensor(label, dtype=torch.long)}

def pad_collate(batch):
    """
    ---- Collate function to pad a list of samples into batched tensors.
    ---- Returns:
    ----- indices_padded: LongTensor (B, L)
    ----- lengths: LongTensor (B,)
    ----- labels: LongTensor (B,)
    ----"""
    batch_sorted = sorted(batch, key=lambda x: x["length"], reverse=True)
    lengths = [item["length"] for item in batch_sorted]
    max_len = lengths[0]
    indices_padded = torch.zeros(len(batch_sorted), max_len, dtype=torch.long)
    labels = torch.zeros(len(batch_sorted), dtype=torch.long)
    for i, item in enumerate(batch_sorted):
        ind = item["indices"]
        indices_padded[i, :len(ind)] = ind
        labels[i] = item["label"]
    return indices_padded, torch.tensor(lengths, dtype=torch.long), labels

def get_dataloaders(cfg, use_local_files: bool = True) -> Dict[str, DataLoader]:
    """
    ---- Build train/val DataLoaders from local files. If local files not present, raise
    ---- Splits data by simple random split.
    ----"""

```



```

cats = load_names_by_category(cfg.data_dir)
if not cats:
    raise FileNotFoundError(f"No category files found in {cfg.data_dir}. Please place the files in the directory.")
# build dataset and split
dataset = NameDataset(cats, max_length=cfg.max_name_length)
n = len(dataset)
# 80/20 split
indices = list(range(n))
np.random.shuffle(indices)
split = int(0.8 * n)
train_idx = indices[:split]
val_idx = indices[split:]
from torch.utils.data import Subset
train_ds = Subset(dataset, train_idx)
val_ds = Subset(dataset, val_idx)
train_loader = DataLoader(train_ds, batch_size=cfg.batch_size, shuffle=True, collate_fn=train_collate)
val_loader = DataLoader(val_ds, batch_size=cfg.batch_size, shuffle=False, collate_fn=val_collate)
return {"train": train_loader, "val": val_loader, "dataset_meta": {"n_classes": n_classes}}

```

Listing 24: data.py

`model.py` contains the neural network definitions used in the assignment. Implementations are modular and parameterized so the same file can be used to instantiate networks used by experiments or tests. This isolates architecture choices from training and experiment orchestration.

```

# model.py
"""
RecurrentClassifier: char-level classifier for names.
Supports GRU / LSTM / vanilla RNN depending on cfg.rnn_type.
"""

import torch
import torch.nn as nn
from typing import Optional

from utils import NLETTERS
import utils as u # to get ALLLETTERS/NLETTERS mapping if needed

VOCAB_SIZE = len(u.ALLLETTERS)

class RecurrentClassifier(nn.Module):
    def __init__(self, n_classes: int, embedding_dim: int = 32, hidden_dim: int = 128,
                rnn_type: str = "gru", num_layers: int = 1, dropout: float = 0.5):
        super().__init__()
        self.embedding = nn.Embedding(VOCAB_SIZE, embedding_dim)

```

```

        self.rnn_type = rnn_type.lower()
        if self.rnn_type == "lstm":
            self.rnn = nn.LSTM(embedding_dim, hidden_dim, num_layers=num_layers,
                                batch_first=True)
        elif self.rnn_type == "gru":
            self.rnn = nn.GRU(embedding_dim, hidden_dim, num_layers=num_layers,
                                batch_first=True)
        else:
            self.rnn = nn.RNN(embedding_dim, hidden_dim, num_layers=num_layers,
                                batch_first=True)
        self.fc = nn.Linear(hidden_dim, n_classes)

    def forward(self, indices: torch.LongTensor, lengths: torch.LongTensor):
        """
        -----indices: (B, L)-long-tensor-of-char-indices-(padded)
        -----lengths: (B,)-long-tensor-of-true-lengths-(descending-order-recommended)
        -----"""
        # embed
        emb = self.embedding(indices)  # (B, L, E)
        # pack
        packed = nn.utils.rnn.pack_padded_sequence(emb, lengths.cpu(), batch_first=True)
        packed_out, hidden = self.rnn(packed)
        # For GRU/RNN/LSTM hidden handling:
        if isinstance(hidden, tuple):  # LSTM returns (h_n, c_n)
            h_n = hidden[0]
        else:
            h_n = hidden
        # h_n: (num_layers, B, hidden_dim)
        last_layer_h = h_n[-1]  # (B, hidden_dim)
        logits = self.fc(last_layer_h)  # (B, n_classes)
        return logits

    def predict(self, indices: torch.LongTensor, lengths: torch.LongTensor) -> torch.Tensor:
        self.eval()
        with torch.no_grad():
            logits = self.forward(indices, lengths)
            preds = torch.argmax(logits, dim=-1)
        return preds

```

Listing 25: model.py

train.py centralizes the training and evaluation logic. It exposes epoch-level functions as well as a high-level **run_training** wrapper that coordinates data loading, model instantiation, optimization, checkpointing, and history logging. By separating epoch logic from orchestration, the code becomes easier to test and reuse.

```

# train.py
"""Training/evaluation loop and helpers. Returns histories akin to the notebook"""

```

```

import time
from typing import Dict, Tuple, Optional
import torch
import torch.nn as nn
from torch.optim import Adam, SGD
from config import Config
from utils import set_seed, configure_logging, save_checkpoint, load_checkpoint
from data import get_dataloaders
from model import BNDropModel, TwoLayerNet

logger = configure_logging()

def build_model(cfg: Config, two_layer: bool = False, weight_scale: float = 1e-2):
    if two_layer:
        return TwoLayerNet(cfg.input_dim, cfg.hidden_dim, cfg.output_dim, weight_scale)
    else:
        return BNDropModel(cfg.input_dim, cfg.hidden_dim, cfg.output_dim, dropout)

def build_optimizer(model: nn.Module, cfg: Config, optim_name: str = "adam"):
    if optim_name.lower() in ("adam", "adamw"):
        return Adam(model.parameters(), lr=cfg.lr, weight_decay=cfg.weight_decay)
    else:
        return SGD(model.parameters(), lr=cfg.lr, weight_decay=cfg.weight_decay)

def train_epoch(model: nn.Module, loader, optimizer, criterion, device: str) -> float:
    model.train()
    total_loss = 0.0
    n = 0
    for X, y in loader:
        X, y = X.to(device), y.to(device)
        optimizer.zero_grad()
        logits = model(X)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * X.size(0)
        n += X.size(0)
    return total_loss / max(1, n)

def eval_epoch(model: nn.Module, loader, criterion, device: str) -> Tuple[float, float]:
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():

```

```

        for X, y in loader:
            X, y = X.to(device), y.to(device)
            logits = model(X)
            loss = criterion(logits, y)
            total += X.size(0)
            total_loss += loss.item() * X.size(0)
            preds = logits.argmax(dim=1)
            correct += (preds == y).sum().item()
        avg_loss = total_loss / max(1, total)
        acc = correct / max(1, total)
    return avg_loss, acc

def run_training(cfg: Config, checkpoint_path: Optional[str] = None, two_layer:
    """Run training, return a dict containing histories and final model+optimizer
    set_seed(cfg.seed)
    device = cfg.device if (cfg.device.startswith("cuda") and torch.cuda.is_avail
    logger.info("Using device: %s", device)

    loaders = get_dataloaders(cfg)
    model = build_model(cfg, two_layer=two_layer, weight_scale=weight_scale).to(
    optimizer = build_optimizer(model, cfg)
    criterion = nn.CrossEntropyLoss()

    if checkpoint_path:
        try:
            ckpt = load_checkpoint(checkpoint_path)
            model.load_state_dict(ckpt.get("model_state", {}))
            optimizer.load_state_dict(ckpt.get("optimizer_state", {}))
            logger.info("Loaded checkpoint %s", checkpoint_path)
        except Exception as e:
            logger.warning("Could not load checkpoint %s: %s", checkpoint_path,

    best_val = float("inf")
    train_losses, val_losses, train_accs, val_accs = [], [], [], []

    for epoch in range(1, cfg.num_epochs + 1):
        t0 = time.time()
        train_loss = train_epoch(model, loaders["train"], optimizer, criterion,
        val_loss, val_acc = eval_epoch(model, loaders["val"], criterion, device)
        elapsed = time.time() - t0
        logger.info("Epoch-%d: train_loss=%0.6f val_loss=%0.6f val_acc=%0.4f (%0.1fs
        train_losses.append(train_loss)
        val_losses.append(val_loss)
        val_accs.append(val_acc)
        if val_loss < best_val:
            best_val = val_loss

```

```

        save_checkpoint({"model_state": model.state_dict(), "optimizer_state": optimizer.state_dict(),
                        "train_losses": train_losses, "val_losses": val_losses, "train_accs": train_accs,
                        "val_accs": val_accs, "model": model, "optimizer": optimizer},
                        logger.info("Saved best checkpoint (val_loss=%.6f) to checkpoints/best_checkpoint.pth"))

    return {
        "train_losses": train_losses,
        "val_losses": val_losses,
        "train_accs": train_accs,
        "val_accs": val_accs,
        "model": model,
        "optimizer": optimizer
    }

```

Listing 26: train.py

`utils.py` contains small helper functions used across the project: seed management, logger configuration, checkpoint helper functions, and simple numeric comparison utilities. These helpers remove boilerplate from the main training and data-loading code.

```

# utils.py
"""Utilities: -seeding, -logging, -checkpointing, -numeric-error-helpers."""
import os
import random
import logging
from typing import Any, Dict, Union
import numpy as np
import torch

def set_seed(seed: int) -> None:
    """Set Python/NumPy/PyTorch-RNG seeds for reproducibility."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

def configure_logging(level: int = logging.INFO) -> logging.Logger:
    """Configure root logger if unset and return a module logger."""
    logger = logging.getLogger()
    if not logger.handlers:
        ch = logging.StreamHandler()
        fmt = "%(asctime)s %(name)s %(levelname)s: %(message)s"
        ch.setFormatter(logging.Formatter(fmt))
        logger.addHandler(ch)
    logger.setLevel(level)
    return logging.getLogger(__name__)

```

```

def save_checkpoint(state: Dict[str, Any], path: str) -> None:
    """Save a PyTorch checkpoint dict to disk, creating dirs as needed."""
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    torch.save(state, path)

def load_checkpoint(path: str, map_location: str = "cpu") -> Dict[str, Any]:
    """Load a PyTorch checkpoint dict."""
    return torch.load(path, map_location=map_location)

# Numeric helpers often used in notebooks (abs/rel errors)
def abs_error(pred: Union[np.ndarray, "torch.Tensor"], target: Union[np.ndarray,
    """Mean absolute error; accepts torch tensors or numpy arrays."""
    if hasattr(pred, "detach"):
        pred = pred.detach().cpu().numpy()
    if hasattr(target, "detach"):
        target = target.detach().cpu().numpy()
    return float(np.mean(np.abs(pred - target)))

def rel_error(pred: Union[np.ndarray, "torch.Tensor"], target: Union[np.ndarray,
    """Relative error: max |pred - target| / max(eps, |target|)."""
    if hasattr(pred, "detach"):
        pred = pred.detach().cpu().numpy()
    if hasattr(target, "detach"):
        target = target.detach().cpu().numpy()
    diff = np.abs(pred - target)
    denom = np.maximum(eps, np.abs(target))
    return float(np.max(diff / denom))

```

Listing 27: utils.py

4 Use of AI Tools

Models: GPT-5

Prompt:

###

< Requirement Description>

Based on the given project documentation and assignment requirements, first summarize the key objectives and coding principles that must be followed.

<Jupyter Notebook>

Then, refactor the provided Jupyter notebook into a clean, modular Python project while keeping all its teaching value.

Step 1: Output a concise bullet list of main requirements and style rules from the document. Step 2: Split the notebook code into several .py files (e.g.,

config.py, data.py, model.py, train.py, utils.py, project.py) with clear headers describing each file's purpose.

Keep all code runnable locally, assume no Google Drive, and cite style or ML engineering references where relevant.

Finally, provide a brief explanation of how each new file maps to the original notebook parts.

###